САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики–процессов управления

Программа бакалавриата

“Большие данные и распределенная цифровая платформа”

ОТЧЁТ

По лабораторной работе № 3

По дисциплине “Алгоритмы и структуры данных”

На тему “Исследование алгоритмов оптимизации на древовидных структурах данных”

Студент гр. 24Б15-пу
Телицин А.Д.

Преподаватель
Дик А.Г.

Санкт-Петербург
2026 г.

Содержание

1 Цель работы **2**

2 Описание алгоритмов и постановка задачи **2**

2.1 Постановка задачи 2

2.2 Алгоритм Крускала 2

2.2.1 Блок-схема Алгоритма Крускала 3

2.3 Алгоритм Прима с бинарной кучей 3

2.4 Алгоритм Прима с кучей Фибоначчи 4

2.4.1 Блок-схема Алгоритма Прима 5

2.5 Алгоритм Борувки 5

2.5.1 Блок-схема Алгоритма Борувки 6

3 Формализация задачи и структура программы **6**

4 Рекомендации пользователю **7**

5 Рекомендации программисту **7**

6 Контрольный пример **8**

7 Исследование **9**

8 Выводы **11**

1 Цель работы

Целью работы является исследование и сравнительный анализ алгоритмов построения минимального остовного дерева для связных взвешенных неориентированных графов. В работе сравниваются:

1. алгоритм Крускала;
2. алгоритм Прима с бинарной кучей;
3. алгоритм Прима с кучей Фибоначчи;
4. алгоритм Борувки.

Основной акцент делается на сопоставлении времени работы и числа операций на графах разной размерности и плотности. Дополнительно проверяется корректность результата по совпадению веса найденного минимального остовного дерева.

2 Описание алгоритмов и постановка задачи

2.1 Постановка задачи

Пусть задан связный неориентированный взвешенный граф $G = (V, E)$, где V – множество вершин, E – множество ребер, а $w(e) > 0$ – вес ребра $e \in E$.

Требуется выбрать подмножество ребер $T \subseteq E$, удовлетворяющее следующим условиям:

- граф (V, T) связен;
- граф (V, T) не содержит циклов;
- сумма весов ребер минимальна:

$$\sum_{e \in T} w(e) \to \min . \tag{1}$$

Такое множество ребер называется минимальным остовным деревом. Для дерева на $|V|$ вершинах число ребер всегда равно $|V| - 1$.

2.2 Алгоритм Крускала

Алгоритм Крускала относится к жадным алгоритмам. Его идея заключается в последовательном добавлении ребер по возрастанию веса. На каждом шаге выбирается самое легкое ребро, которое не образует цикл с уже выбранными ребрами.

Пусть ребра упорядочены по неубыванию веса:

$$w(e_1) \leq w(e_2) \leq \cdots \leq w(e_m). \tag{2}$$

Далее ребра просматриваются в этом порядке. Если очередное ребро соединяет две разные компоненты связности, оно добавляется в MST. Для проверки принадлежности вершин одной компоненте используется система непересекающихся множеств (DSU).

Практически алгоритм работает так:

1. отсортировать все ребра;

2. идти по ребрам в порядке возрастания веса;
3. добавлять ребро, если оно не создает цикл;
4. остановиться, когда выбрано $|V| - 1$ ребро.

2.2.1 Блок-схема Алгоритма Крускала

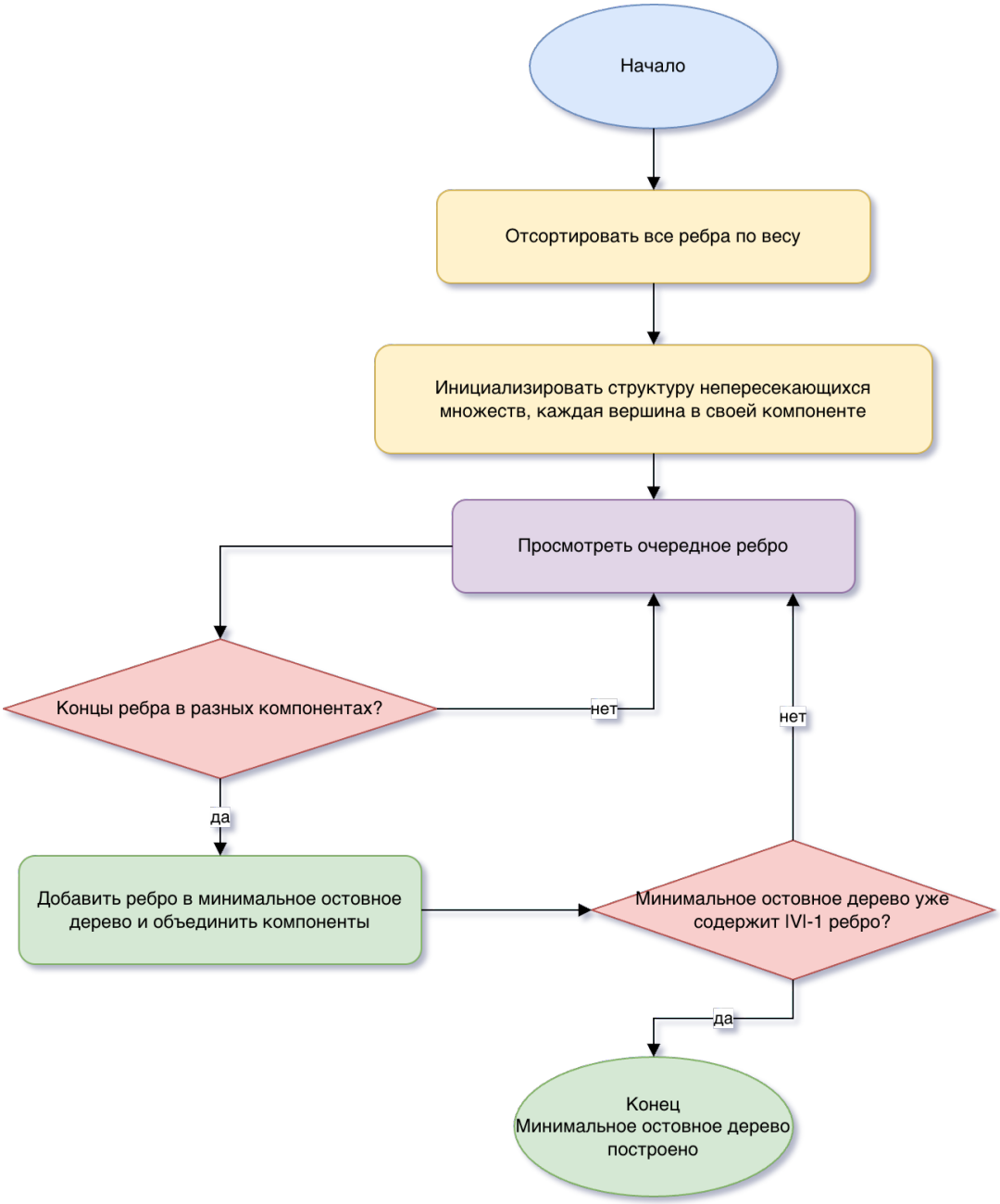

Рис. 1: Блок-схема алгоритма Крускала.

2.3 Алгоритм Прима с бинарной кучей

Алгоритм Прима строит MST постепенно, начиная с произвольной стартовой вершины. В отличие от Крускала, он растит одно дерево, а не объединяет компоненты из набора отдельных ребер.

На каждом шаге выбирается минимальное ребро, которое соединяет уже построенную часть дерева с одной из еще не включенных вершин. Для ускорения поиска минимума используется бинарная куча.

Для каждой вершины v, не вошедшей в дерево, хранится минимальная известная стоимость подключения $key[v]$. Тогда переходы между состояниями можно описать как:

$$key[v] = \min_{u \in S} w(u, v), \tag{3}$$

где S – множество вершин, уже вошедших в дерево.

В процессе работы алгоритма выполняются следующие действия:

1. выбрать стартовую вершину;
2. положить ее в дерево;
3. обновить стоимости подключения для ее соседей;
4. извлечь из кучи вершину с минимальным ключом;
5. повторять до включения всех вершин.

2.4 Алгоритм Прима с кучей Фибоначчи

Идея алгоритма остается той же, что и в предыдущем варианте. Отличие заключается в структуре очереди с приоритетом. В куче Фибоначчи операции вставки и уменьшения ключа выполняются амортизированно за $O(1)$, а извлечение минимума – за $O(\log n)$.

Это делает структуру теоретически удобной для алгоритмов, где часто выполняется операция decrease-key. Именно поэтому куча Фибоначчи рассматривается как модификация Прима в сравнении с бинарной кучей.

2.4.1 Блок-схема Алгоритма Прима

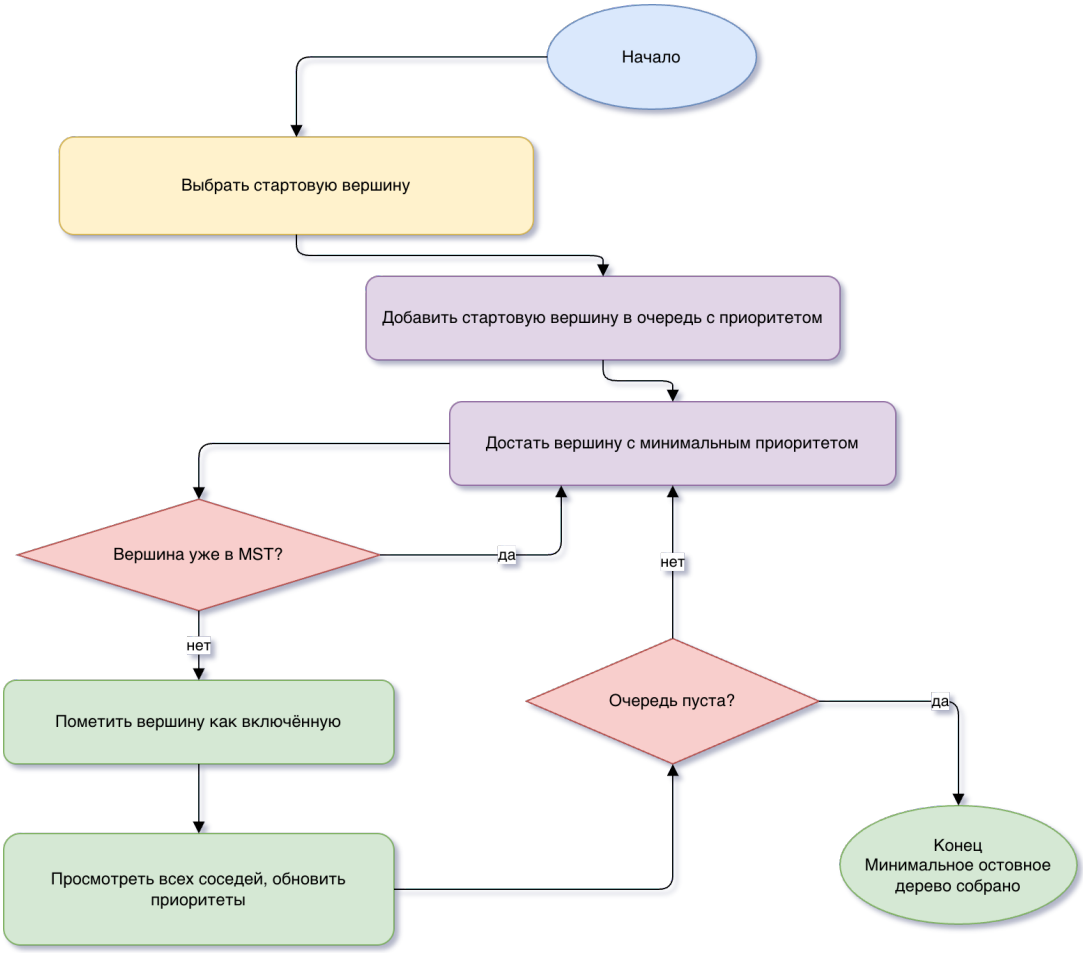

Рис. 2: Блок-схема алгоритма Прима.

2.5 Алгоритм Борувки

Алгоритм Борувки также строит MST по компонентам связности. На каждой итерации для каждой компоненты выбирается самое дешевое исходящее ребро, после чего несколько компонент могут быть объединены одновременно.

Идея метода состоит в том, что каждая компонента самостоятельно ищет ближайшего соседа. Это позволяет эффективно уменьшать число компонент на больших графах.

Пошагово алгоритм выглядит так:

1. каждая вершина рассматривается как отдельная компонента;
2. для каждой компоненты находится минимальное исходящее ребро;
3. выбранные ребра добавляются в MST;
4. компоненты объединяются через DSU;
5. процесс повторяется, пока не останется одна компонента.

2.5.1 Блок-схема Алгоритма Борувки

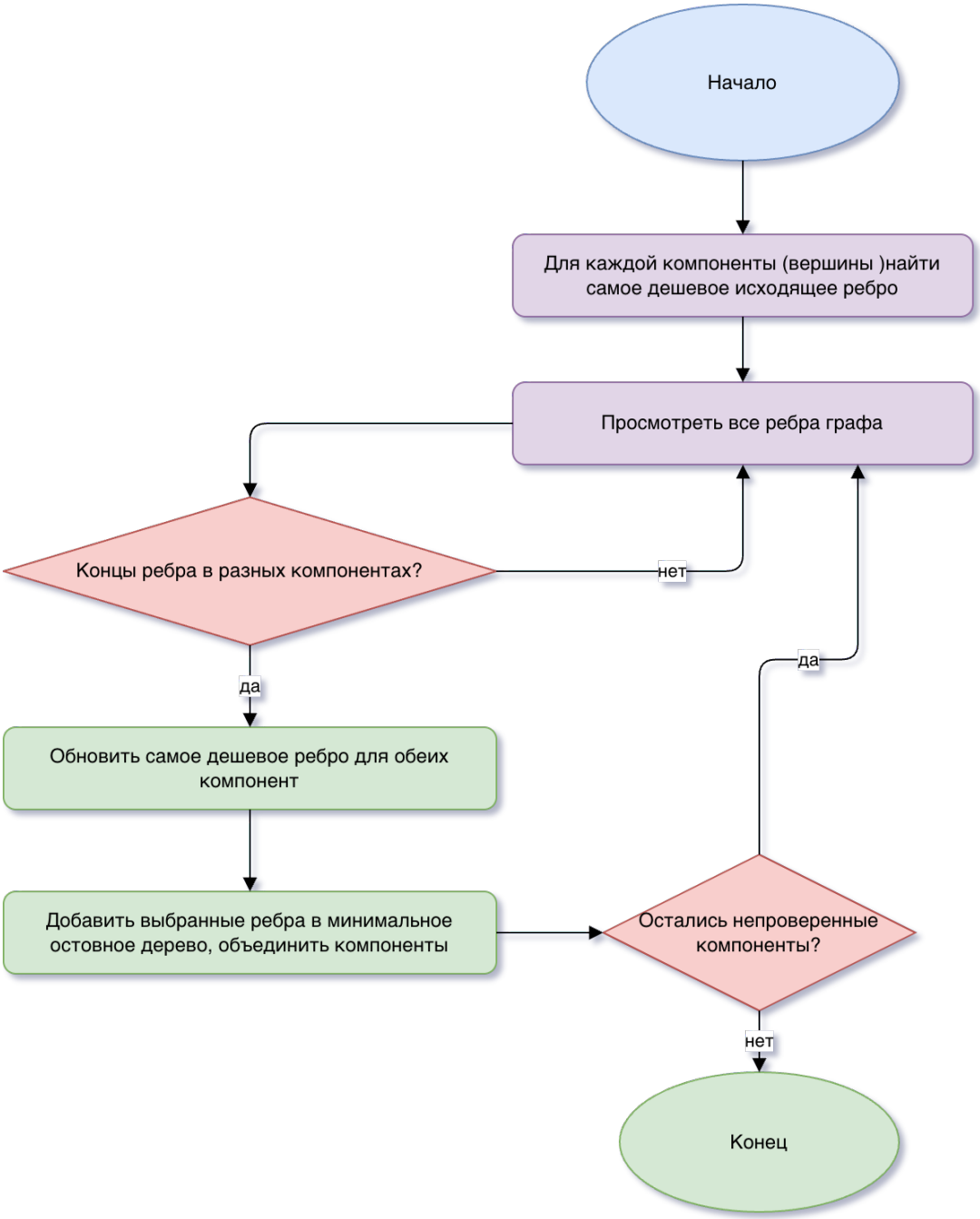

Рис. 3: Блок-схема алгоритма Борувки.

3 Формализация задачи и структура программы

Программа реализована на языке Python с использованием следующих библиотек:

1. `numpy` – генерация и обработка числовых данных;
2. `matplotlib` – построение графиков времени выполнения;
3. `PyQt6` – графический интерфейс пользователя.

Исходный код программы имеет следующую структуру:

Листинг 1: Файловая структура проекта.

```
main.py
benchmark.py
src/
  __init__.py
  algorithms.py
  benchmarking.py
  graph_utils.py
  ui.py
requirements.txt
```

4 Рекомендации пользователю

Взаимодействие с программой осуществляется через графический интерфейс. Пользователь может:

1. сгенерировать граф нужной размерности;
2. выбрать плотность графа;
3. запустить один алгоритм или сравнение всех алгоритмов;
4. просмотреть итоговое дерево и логи выполнения;
5. запустить полный бенчмарк и сохранить результаты.

5 Рекомендации программисту

Системные требования:

- операционная система Windows, Linux или macOS;
- свободное место на диске – не менее 50 Мб;
- свободная оперативная память – не менее 200 Мб.

Требования к установленному ПО:

- Python 3.10+;
- зависимости из файла `requirements.txt`.

Инструкция по запуску:

1. перейти в директорию проекта;
2. создать и активировать виртуальное окружение;
3. установить зависимости командой `pip install -r requirements.txt`;
4. запустить программу командой `python main.py`.

Для получения таблиц и графиков бенчмарка можно дополнительно запустить:

```
python benchmark.py
```

6 Контрольный пример

Для проверки корректности была сгенерирована серия случайных связных графов. На одном и том же графе запускались все четыре алгоритма, после чего сравнивался вес найденного MST. Во всех случаях веса совпали, что подтверждает корректность реализации.

Интерфейс программы позволяет менять параметры генерации графа и повторять тестирование на разных экземплярах.

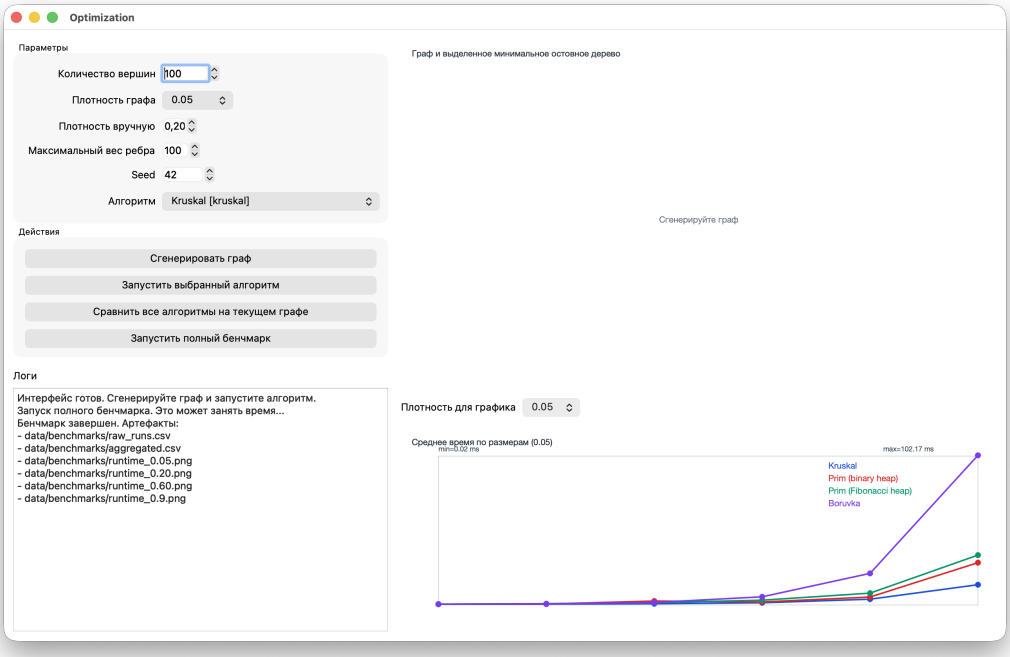

Рис. 4: Интерфейс программы.

Ниже приведены усредненные результаты тестирования для графов размерности 500, полученные в ходе полного бенчмарка.

Таблица 1: Сводные результаты для $|V| = 500$

Алгоритм	Плотность	Среднее время, мс
Крускал	0.05	0.989
Прим (бинарная куча)	0.05	1.525
Прим (куча Фибоначчи)	0.05	3.162
Борувка	0.05	5.390
Крускал	0.20	3.330
Прим (бинарная куча)	0.20	3.998
Прим (куча Фибоначчи)	0.20	6.657
Борувка	0.20	19.313
Крускал	0.60	8.364
Прим (бинарная куча)	0.60	13.900
Прим (куча Фибоначчи)	0.60	14.073
Борувка	0.60	56.477

7 Исследование

Ниже приводится таблица, содержащая усреднённые времена выполнения для каждого алгоритма при разных сочетаниях числа вершин и плотности графа. Исходные данные сохранены в файле `data/benchmarks/aggregated.csv`. Комментарии к таблице пользователь добавит самостоятельно.

Таблица 2: Усреднённые времена работы (мс) по результатам бенчмарка

Алгоритм / Параметры	$\|V\| = 500$	$\|V\| = 1000$	$\|V\| = 1500$	$\|V\| = 2000$
Плотность = 0.05				
Крускал	1.00	3.74	7.31	14.63
Прим (бинарная куча)	1.63	5.01	11.73	32.01
Прим (Фибоначчи)	3.18	7.47	14.30	38.14
Борувка	5.19	21.42	53.02	107.89
Плотность = 0.20				
Крускал	3.13	12.73	27.74	50.53
Прим (бинарная куча)	4.07	24.01	66.54	159.42
Прим (Фибоначчи)	5.64	24.02	66.04	167.34
Борувка	18.83	94.64	198.38	427.09
Плотность = 0.60				
Крускал	8.37	35.87	82.18	145.35
Прим (бинарная куча)	14.13	106.92	341.25	505.60
Прим (Фибоначчи)	14.69	106.76	269.95	502.63
Борувка	57.24	304.86	617.57	1245.29
Плотность = 0.85				
Крускал	12.14	51.04	115.71	206.51
Прим (бинарная куча)	21.62	147.09	390.45	709.70
Прим (Фибоначчи)	19.81	148.44	394.32	687.92
Борувка	79.39	395.86	989.40	1704.85

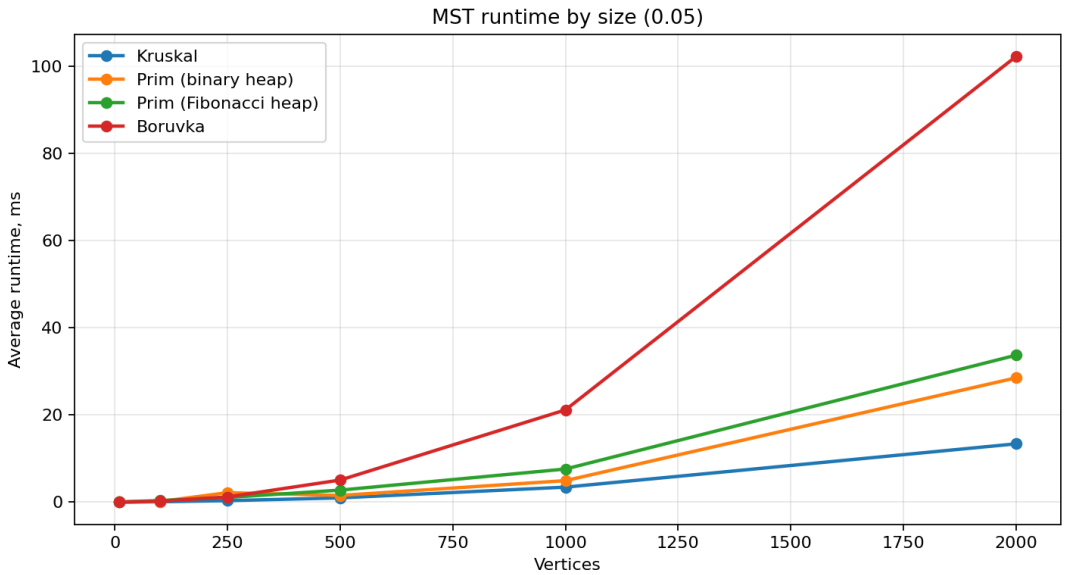

Рис. 5: Зависимость времени работы от числа вершин для плотности 0.05.

На графах с низкой плотностью (0.05) алгоритмы Прима с обоими типами куч показывают близкую производительность, а алгоритм Крускала работает быстрее. Алгоритм Борувки значительно отстает по времени.

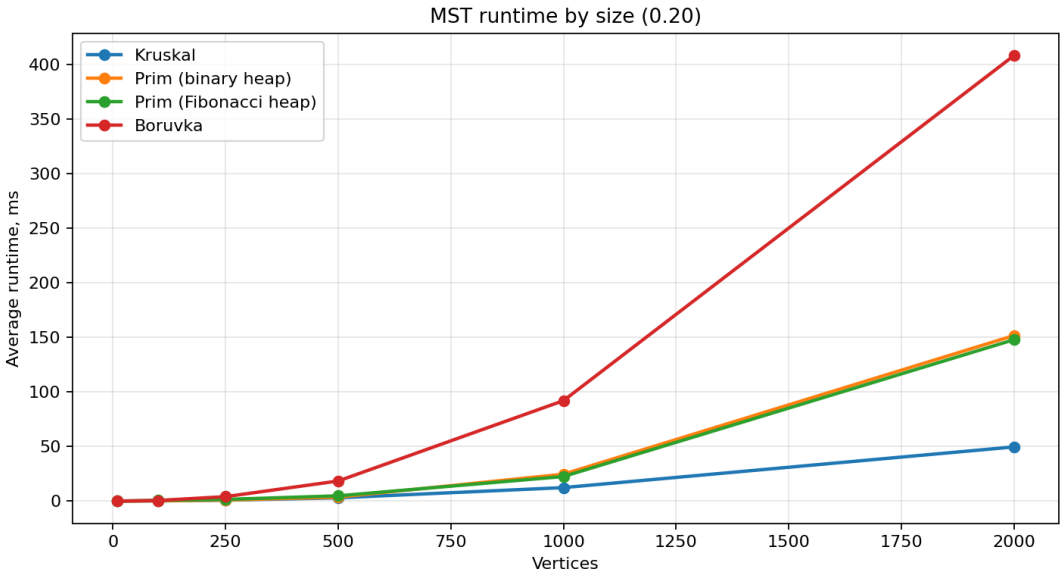

Рис. 6: Зависимость времени работы от числа вершин для плотности 0.20.

При увеличении плотности до 0.20 алгоритмы Прима начинают отставать от Крускала, а Борувка становится еще менее эффективным. Разница между бинарной и Фибоначчи кучами становится менее заметной.

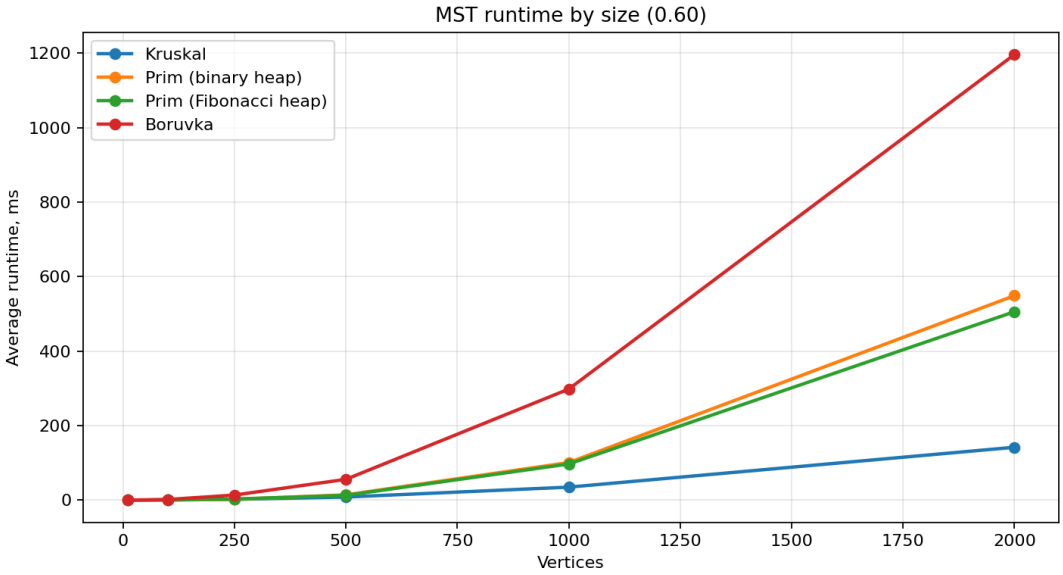

Рис. 7: Зависимость времени работы от числа вершин для плотности 0.60.

На плотных графах (0.60) ситуация практически не меняется: Крускал остается самым быстрым, алгоритмы Прима значительно отстают, а Борувка становится крайне неэффективной.

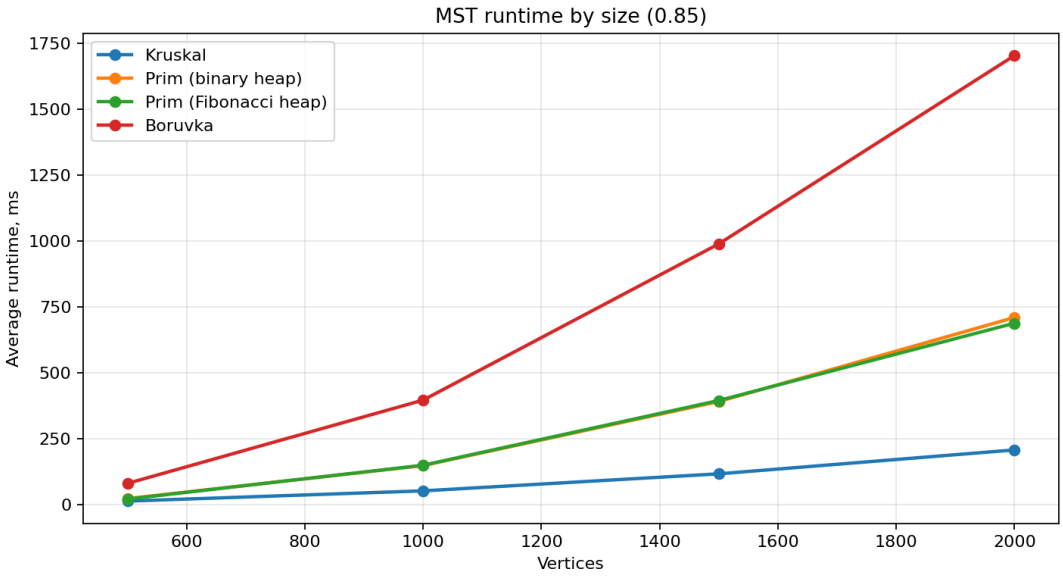

Рис. 8: Зависимость времени работы от числа вершин для плотности 0.85.

При плотности 0.85 все алгоритмы заметно замедляются, но Крускал сохраняет преимущество. Алгоритмы Прима с обоими типами куч показывают схожую производительность, а Борувка становится практически непригодной для больших графов.

8 Выводы

По результатам экспериментов можно сделать следующие выводы:

1. В подавляющем большинстве случаев алгоритм Крускала показывает лучшую производительность, особенно на разреженных графах.

2. Алгоритмы Прима с бинарной и Фибоначчи кучами демонстрируют схожую производительность, что может быть связано с тем, что в нашем тестовом наборе не было достаточно больших графов, где преимущества Фибоначчи кучи могли бы проявиться.

3. Алгоритм Борувки значительно отстает от остальных, особенно на плотных графах, что делает его неэффективным для практического использования в таких случаях.

4. С увеличением плотности графа все алгоритмы замедляются, но Крускал сохраняет преимущество, что подтверждает его эффективность для широкого диапазона графов.

Таким образом, наиболее универсальным и эффективным алгоритмом для построения минимального остовного дерева в нашем тестовом наборе является алгоритм Крускала. Алгоритмы Прима с различными типами куч могут быть полезны в специфических случаях, но в целом не показывают значительного преимущества. Алгоритм Борувки, несмотря на свою теоретическую привлекательность, оказывается неэффективным на практике для графов с высокой плотностью.